

Bloque 4: Consultas con JPA / Hibernate

Objetivo del bloque

Aprender a realizar consultas más allá de `find()`, entendiendo cómo se recuperan datos reales usando JPA y cómo se traduce esto a SQL internamente.

1. Limitación de find()

Hasta ahora se ha usado:

```
Usuario u = em.find(Usuario.class, 1);
```

Esto solo permite:

- Buscar por clave primaria

Problema:

En aplicaciones reales se necesitan consultas como:

- Buscar usuarios por nombre
- Filtrar por fechas
- Obtener listas
- Combinar datos de varias entidades

Para eso se utilizan consultas en JPA.

2. JPQL (Java Persistence Query Language)

JPQL es un lenguaje de consultas similar a SQL, pero orientado a objetos.

Diferencia clave:

- SQL trabaja con tablas y columnas
 - JPQL trabaja con entidades y atributos
-

Ejemplo comparativo

SQL:

```
SELECT * FROM usuarios WHERE nombre = 'Ana';
```

JPQL:

```
SELECT u FROM Usuario u WHERE u.nombre = 'Ana'
```

Diferencias:

- Se usa la clase Usuario (no la tabla)
 - Se usa u.nombre (no la columna directamente)
 - Se trabaja con objetos
-

3. Crear consultas con EntityManager

Query básica

```
Query query = em.createQuery("SELECT u FROM Usuario u");  
List<Usuario> usuarios = query.getResultList();
```

Esto devuelve una lista de objetos Usuario.

4. TypedQuery (forma recomendada)

Permite evitar casting.

```
TypedQuery<Usuario> query =  
    em.createQuery("SELECT u FROM Usuario u", Usuario.class);  
  
List<Usuario> usuarios = query.getResultList();
```

Ventaja:

- Tipado seguro
 - Código más limpio
-

5. Parámetros en consultas

Evitan concatenación de strings y problemas de seguridad.

```
TypedQuery<Usuario> query =  
    em.createQuery("SELECT u FROM Usuario u WHERE u.nombre = :nombre",  
        Usuario.class);
```

```
query.setParameter("nombre", "Ana");
```

```
List<Usuario> usuarios = query.getResultList();
```

Idea clave:

Siempre usar parámetros en lugar de concatenar valores.

6. Consultas con condiciones

Ejemplo:

```
SELECT u FROM Usuario u WHERE u.email LIKE :email
```

Uso:

```
query.setParameter("email", "%gmail.com");
```

7. Consultas con relaciones (JOIN)

Aquí se ve el verdadero poder del ORM.

Ejemplo: obtener pedidos con su usuario

```
SELECT p FROM Pedido p JOIN p.usuario u WHERE u.nombre = :nombre
```

Explicación:

- No se hace JOIN sobre tablas
 - Se hace JOIN sobre relaciones entre objetos
-

Ejemplo más completo

```
TypedQuery<Pedido> query =  
    em.createQuery(  
        "SELECT p FROM Pedido p JOIN p.usuario u WHERE u.nombre = :nombre",  
        Pedido.class  
    );
```

```
query.setParameter("nombre", "Ana");
```

```
List<Pedido> pedidos = query.getResultList();
```

8. SELECT parcial

Se pueden seleccionar campos concretos:

```
SELECT u.nombre FROM Usuario u
```

Resultado:

- Lista de Strings (no objetos completos)
-

También se pueden devolver varios campos:

```
SELECT u.nombre, u.email FROM Usuario u
```

Resultado:

- Lista de Object[]
-

9. Named Queries

Permiten definir consultas reutilizables en la entidad.

```
@Entity
@NamedQuery(
    name = "Usuario.buscarPorNombre",
    query = "SELECT u FROM Usuario u WHERE u.nombre = :nombre"
)
public class Usuario {
    ...
}
```

Uso:

```
TypedQuery<Usuario> query =
    em.createNamedQuery("Usuario.buscarPorNombre", Usuario.class);

query.setParameter("nombre", "Ana");

List<Usuario> usuarios = query.getResultList();
```

Ventaja:

- Reutilización
- Centralización de consultas

10. Introducción a Criteria API

Es una forma de construir consultas de manera programática (sin strings).

Ejemplo básico:

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Usuario> cq = cb.createQuery(Usuario.class);

Root<Usuario> root = cq.from(Usuario.class);
cq.select(root);

TypedQuery<Usuario> query = em.createQuery(cq);
List<Usuario> usuarios = query.getResultList();
```

Características

- Evita errores de sintaxis en strings
- Más seguro en tiempo de compilación
- Más complejo de leer

Idea clave:

Se usa en consultas dinámicas, pero JPQL es más habitual.

11. Qué ocurre internamente

Cuando se ejecuta una consulta JPQL:

```
SELECT u FROM Usuario u WHERE u.nombre = :nombre
```

Hibernate la transforma en SQL:

```
SELECT id, nombre, email
FROM usuarios
WHERE nombre = 'Ana';
```

Y luego:

- Ejecuta la consulta
 - Convierte resultados en objetos
-

12. Problemas comunes en consultas

12.1 Pensar en tablas en lugar de objetos

Error típico:

Intentar escribir SQL en lugar de JPQL.

12.2 No usar parámetros

Puede provocar:

- errores
 - problemas de seguridad
-

12.3 Consultas ineficientes

Especialmente con relaciones (se verá en el siguiente bloque).

13. Idea final del bloque

JPA no elimina las consultas, las transforma.

Se deja de escribir SQL directamente, pero:

- Se sigue consultando datos
- Se sigue necesitando entender cómo funcionan las consultas

La diferencia es que ahora:

- Se trabaja con entidades
- Se usan relaciones en lugar de joins manuales
- El ORM se encarga de traducir todo a SQL automáticamente

Este bloque es el paso necesario para trabajar con datos reales en aplicaciones complejas.

14. Ejercicio de ejemplo

```
import jakarta.persistence.*;
import java.util.List;

public class Main {

    public static void main(String[] args) {

        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("miUnidad");

        EntityManager em = emf.createEntityManager();

        // Buscar usuarios por nombre
        TypedQuery<Usuario> query =
            em.createQuery(
                "SELECT u FROM Usuario u WHERE u.nombre = :nombre",
                Usuario.class
            );

        query.setParameter("nombre", "Ana");

        List<Usuario> usuarios = query.getResultList();

        for (Usuario u : usuarios) {
            System.out.println(u.getNombre());
        }

        // JOIN con pedidos
        TypedQuery<Pedido> query2 =
            em.createQuery(
                "SELECT p FROM Pedido p JOIN p.usuario u WHERE u.nombre = :nombre",
                Pedido.class
            );

        query2.setParameter("nombre", "Ana");

        List<Pedido> pedidos = query2.getResultList();

        for (Pedido p : pedidos) {
            System.out.println(p.getDescripcion());
        }

        em.close();
        emf.close();
    }
}
```


EJERCICIOS

1. **Buscar usuarios por nombre (JPQL + parámetros)**
2. **Buscar productos con precio mayor a X**
3. **Obtener pedidos por fecha**
4. **JOIN usuario → pedidos**
5. **JOIN pedido → productos**
6. **Consultar solo nombres de usuarios (SELECT parcial)**
7. **Consulta con LIKE (emails gmail)**
8. **NamedQuery en Usuario y ejecución**
9. **Criteria API: listar todos los usuarios**
10. **Criteria API: filtro por nombre dinámico**